

C Programming

File Input/Output: Ch.15

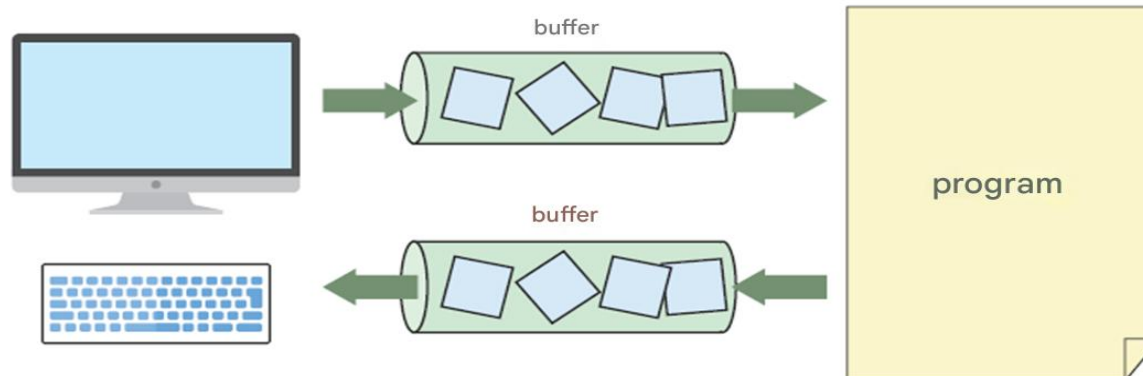
Streams | Text vs. binary | Opening/closing | Reading & writing

What You Will Learn in This Chapter

- The concept of streams
- **Standard Input**/Output
- File Input/Output
- Input/output related functions

The Concept of a Stream

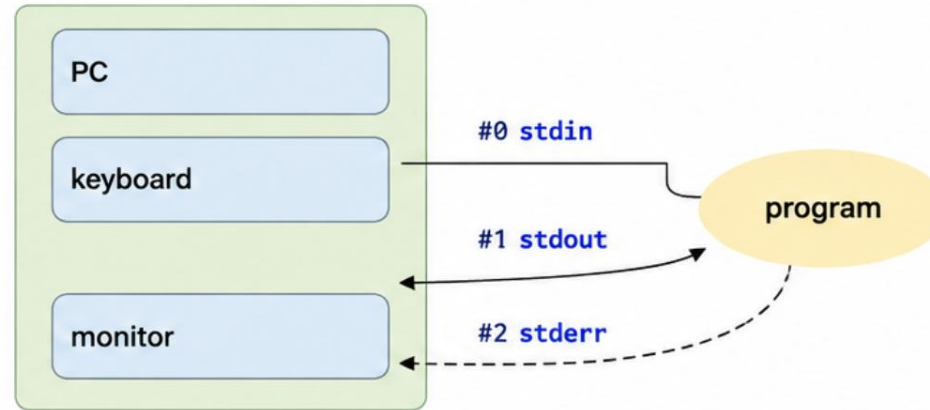
- **Stream**: thinking of input and output as a flow of bytes - a data path.
- A stream uses a buffer by default in C.
- Stream → Buffer → File / Device
 - Stream: data flow between a program and a file/device
 - Buffer: temporary memory used to make I/O more efficient
 - Most streams use buffering, but buffering behavior can vary. For example, stderr is typically unbuffered.



C Input/Output: Standard Streams & Classification of I/O Functions

Standard Streams in C

name	stream	connection
stdin	standard input stream	keyboard
stdout	standard output stream	monitor screen
stderr	standard error stream	monitor screen



Classification of Input/Output Functions

form \ stream	standard stream	normal stream (FILE *f,...)	function	meaning
Unformatted I/O (characters)	getchar()	fgetc(FILE *f,...)	Character input function	Read one character
	putchar()	fputc(FILE *f,...)	Character output function	Write one character
	gets_s()	fgets(FILE *f,...)	String input function	Read a line (string)
	puts()	fputs(FILE *f,...)	String output function	Write a line (string)
Formatted I/O (integer, real number, etc)	printf()	fprintf(FILE *f,...)	Formatted output function	Print formatted data
	scanf()	fscanf(FILE *f,...)	Formatted input function	Read formatted data



stdin (keyboard)
input from the user



stdout (monitor)
normal program output

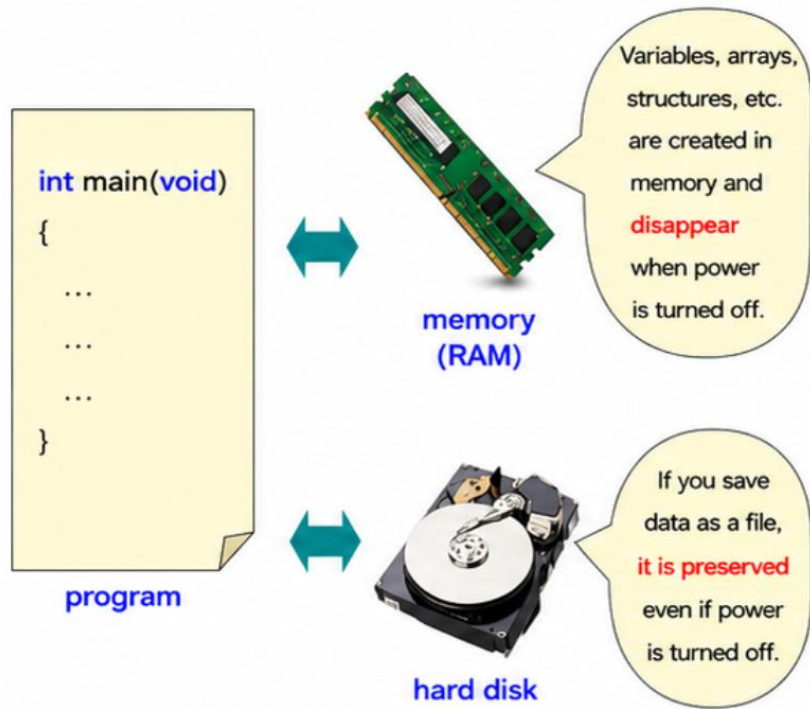


stderr (monitor)
error messages

#0 = stdin, #1 = stdout, #2 = stderr
These are standard stream numbers.

File in C

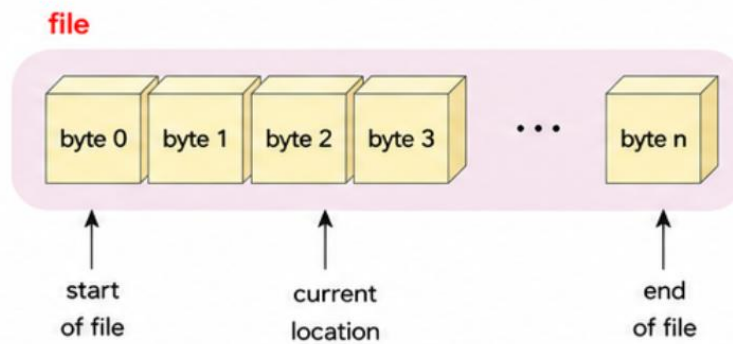
1. Why do I need files?



Files store data permanently.
Programs can save data and read it back later.

2. The concept of a file

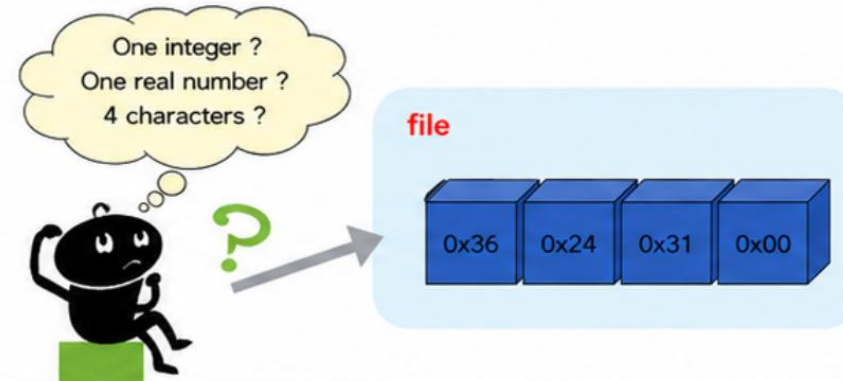
- A file in C is a series of contiguous bytes.
- All file data is eventually converted to bytes and stored in a file.
- It is entirely up to the programmer to **interpret** these bytes.



A file = a sequence of bytes with a beginning and an end.

3. File (Text file & Binary file)

- If a file contains 4 bytes, it can be interpreted as either integer data (**int**) or real number data (**float**).



Text file

- Bytes are interpreted as characters.
- Example: "A", "B", "1" (stored as ASCII codes)
- Human-readable.

0x41	0x42	0x31	0x0A
A	B	1	\n

Binary file

- Bytes are interpreted as numbers (int, float, etc.).
- Example: integer 12345 (stored as binary bytes)
- Not human-readable.

0x39	0x30	0x00	0x00
(int) 12345			



Summary



Memory is volatile.



Files store data permanently on storage devices.



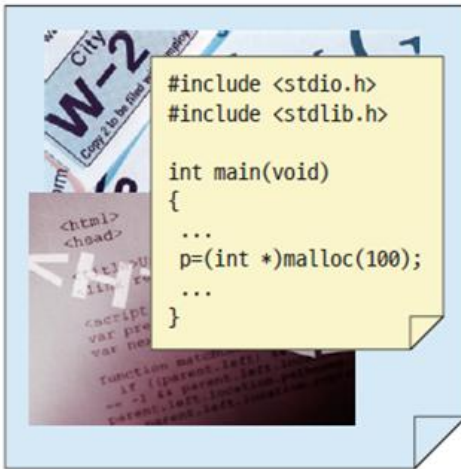
A file is a sequence of bytes.



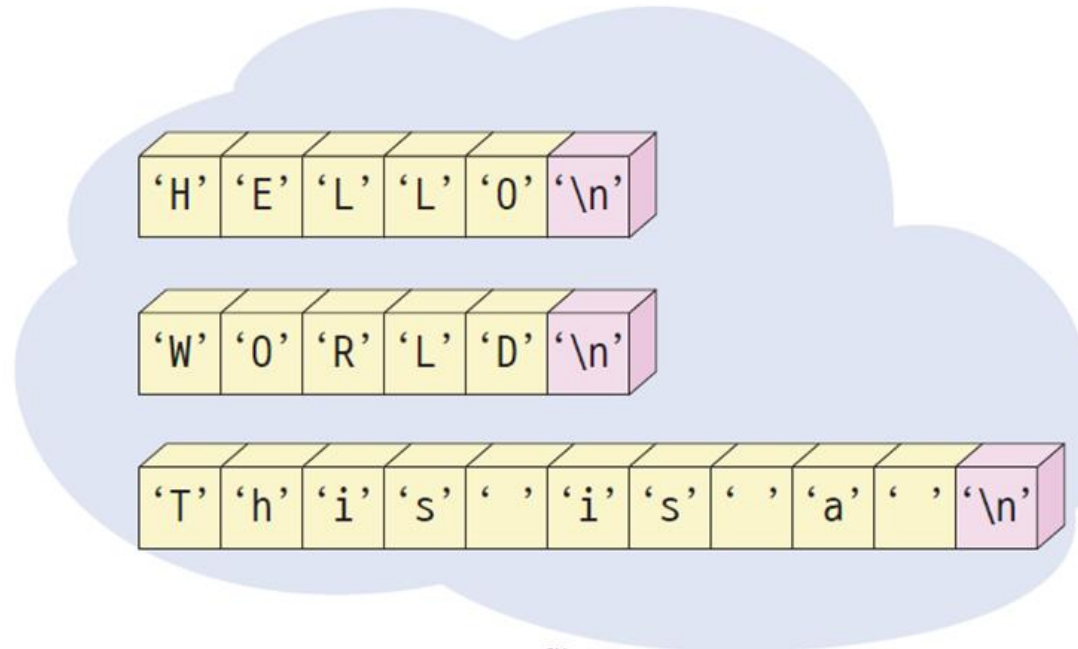
Same bytes can be interpreted as text or binary data.

File in C: Text File

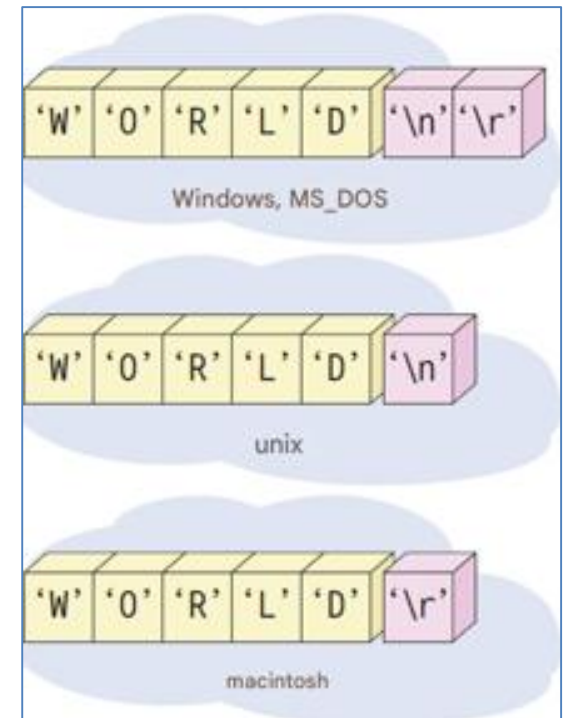
- **Text file:** a file that contains human-readable text.
(example: a C program source file, or a Notepad file).
- Text files are saved using ASCII codes.
 - Each operating system has a different way of displaying line breaks.



Text_file: a file consisting of characters

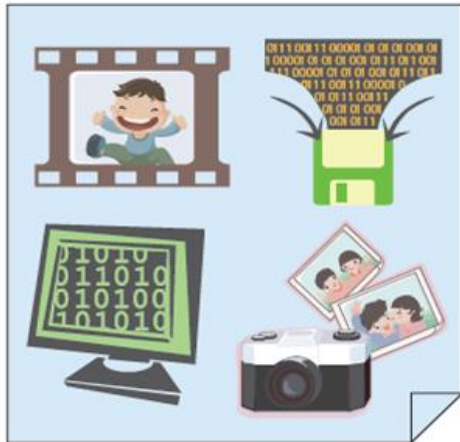


text file

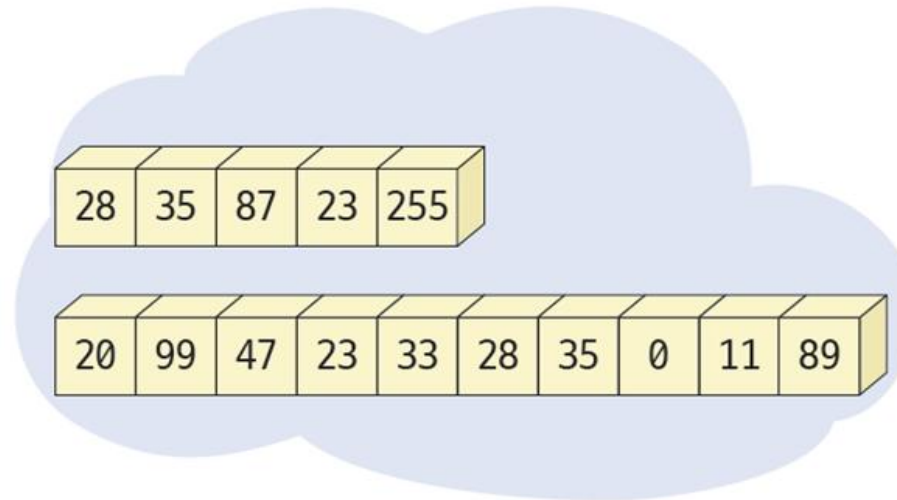


File in C: Binary File

- **Binary file:** a file that cannot be read by humans but can be read by computers
 - a file that directly stores binary data.
- Unlike text files, binary files are not separated into lines; all data is input/output without being converted to strings. (C program executable file, a sound file, an image file)



Binary file: A file consisting of data.

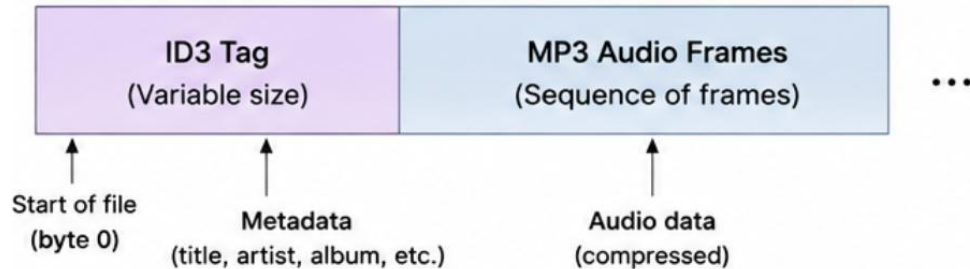


binary file

Binary File: MP3 Header Information

An MP3 file typically begins with an ID3 tag (metadata) followed by audio data frames. Each audio frame has its own header that describes how the audio data is encoded.

1. Overall Structure of an MP3 File



2. ID3 Tag Header (ID3v2)

Byte(s)	0-2	3	4	5	6	7-9
Size	3 bytes	1 byte	1 byte	1 byte	1 byte	4 bytes
Field	ID3	Version	Revision	Flags	Size (synchsafe int)	

- "ID3" : Identifier (ASCII "ID3")
- Version : ID3 version (e.g., 03 for ID3v2.3, 04 for ID3v2.4)
- Revision : Revision number
- Flags : Various options (e.g., unsynchronisation, extended header)
- Size : Size of the ID3 tag (excluding the 10-byte header) stored as synchsafe integer (7 bits per byte)



ID3 Tag Contains (Examples)

Title, Artist, Album, Year, Genre, Comment, Track number, etc.

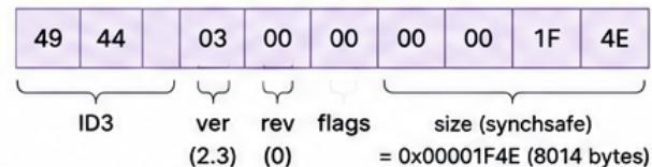
3. MP3 Frame Header (4 bytes)



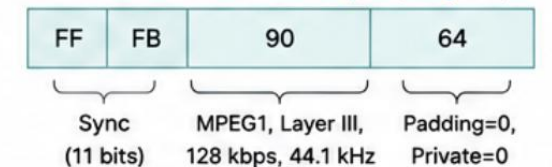
Field	Description
Sync (11 bits)	Frame sync. Always 1 (binary: 11111111111)
MPEG Version (2 bits)	00: MPEG 2.5, 01: Reserved, 10: MPEG 2, 11: MPEG 1
Layer (2 bits)	00: Reserved, 01: Layer III, 10: Layer II, 11: Layer I
Protection (1 bit)	0: Protected (has CRC), 1: Not protected
Bitrate Index (4 bits)	Indicates the bitrate (see bitrate table)
Sample Rate Index (2 bits)	Indicates the sampling rate (see sample rate table)
Padding (1 bit)	0: No padding, 1: Add one slot of padding
Private (1 bit)	Used by encoder, not used by decoder

4. Example

ID3v2.3 Header (10 bytes)



MP3 Frame Header Example (4 bytes)



Binary File: JPEG Header Information

A JPEG file consists of a sequence of **markers** (segments).

The header contains information about the image size, color, compression, and other parameters.

1. Overall Structure of a JPEG File



SOI : Start Of Image (marker: 0xFFD8)
APPn : Application segment (e.g., JFIF: 0xFFE0)
DQT : Define Quantization Table (0xFFDB)
SOF0 : Start Of Frame (Baseline DCT) (0xFFC0)

DHT : Define Huffman Table (0xFFC4)
SOS : Start Of Scan (0xFFDA)
EOI : End Of Image (0xFFD9)

2. Common JPEG Markers

Marker	Hex Code	Name	Description
SOI	FFD8	Start Of Image	Marks the beginning of the JPEG file
APP0	FFE0	JFIF Header	Contains JFIF information (version, density, etc.)
DQT	FFDB	Define Quantization Table	Defines one or more quantization tables
SOF0	FFC0	Start Of Frame (Baseline DCT)	Specifies image size , precision , number of components
DHT	FFC4	Define Huffman Table	Defines Huffman coding tables
SOS	FFDA	Start Of Scan	Indicates start of compressed image data
EOI	FFD9	End Of Image	Marks the end of the JPEG file

3. APP0 (JFIF) Segment Structure

Marker	Length	Identifier "JFIF"	Version Major	Version Minor	Units	Xdensity	Ydensity	Xthumb	Ythumb	Thumbnail (if any)
2 bytes	2 bytes	5 bytes	1 byte	1 byte	2 bytes	2 bytes	2 bytes	1 byte	1 byte	3 bytes

Example (no thumbnail)

FF	D8	FF	E0	00	10	4A	46	49	46	00	01	02	01	00	48	00	48	00	00
SOI		APP0		Length (16 bytes)		Identifier "JFIF"		Ver Major	Ver Minor	Units (1: dots/inch)	Xdensity (72)	Ydensity (72)	Xthumb (0)	Ythumb (0)					

4. SOF0 (Start Of Frame) Segment Structure (Baseline DCT)

Marker (0xFFC0)	Length	Sample Precision	Image Height	Image Width	Number of Components
2 bytes	2 bytes	1 byte	2 bytes	2 bytes	1 byte

Followed by (for each component)

Component ID	Sampling Factors (H, V)	Quantization Table Selector
1 byte	1 byte	1 byte

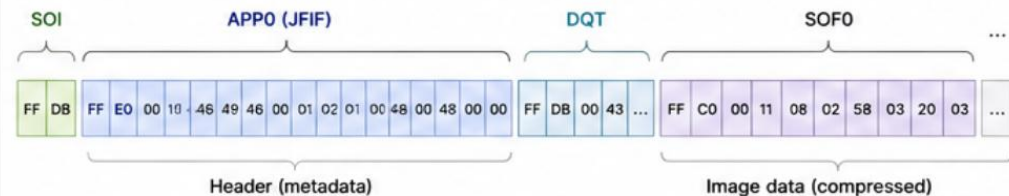
Example: 800 x 600, 3 components (Y, Cb, Cr)

FF	C0	00 11	08	02	58	03	20	03
01	22	00	02	11	01	03	11	01

FF C0 : SOF0 marker
00 11 : Length = 17 bytes
08 : Sample precision = 8 bits
02 58 : Height = 0x0258 = 600
03 20 : Width = 0x0320 = 800
03 : Number of components = 3

22 : Sampling factor = 0
02 : Component ID = 2 (Cb)
11 : Sampling factors = H:1, V:1
01 : Quantization table = 1
03 : Component ID = 3 (Cr)
11 : Sampling factors = H:1, V:1
01 : Quantization table = 1

5. Example: Beginning of a JPEG File (Hex Dump)



Overview of File Processing

- When handling files, you must follow this order:
open the file, use it (**read/write**), then **close** it.



- Disk files are accessed using the FILE structure.
A **file pointer** is a pointer to a FILE structure.
- fopen()** creates/opens a file with the given file name and returns a **FILE pointer**.

```
struct _iobuf {  
    char * _ptr ;  
    int _cnt ;  
    char * _base;  
    int _flag;  
    int _file;  
    int _charbuf ;  
    int _bufsiz ;  
    char * _tmpfname ;  
};  
typedef struct _iobuf FILE;
```

Opening and Closing a File: Text vs. Binary

1. Open File (fopen)

Syntax open file

```
FILE *fp;  
fp = fopen("test.txt", "w");
```

file name file mode

Text Mode (for text files)

Mode	Meaning	If file exists	If file does not exist
r	Read from the beginning	Open for reading	Fail (NULL)
w	Write from the beginning	Open for writing (truncate / erase)	Create new file for writing
a	Append (write from the end)	Open and write at the end	Create new file for writing

Binary Mode (for binary files)

Mode	Meaning	If file exists	If file does not exist
rb	Read from the beginning	Open for reading	Fail (NULL)
wb	Write from the beginning	Open for writing (truncate / erase)	Create new file for writing
ab	Append (write from the end)	Open and write at the end	Create new file for writing

 **Text mode:** Data is treated as characters. Newline (\n) may be translated to a platform-specific format.

Binary mode: Data is treated as raw bytes. No translation is performed.

2. Close File (fclose)

Syntax close file

```
fclose(fp);
```

FILE pointer

✓ Notes

- Always open a file before use and close it after use.
- fopen() returns a FILE pointer; fclose() releases the file and associated resources.
- If fopen() fails, it returns **NULL**. Always check before use.

Opening and Closing a File

```
#include < stdio.h >
int main( void )
{
    FILE * fp = NULL;

    fp = fopen ( "sample.txt" , "w" );

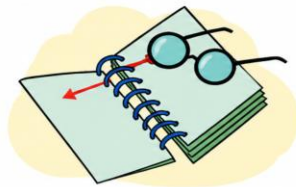
    if ( fp == NULL )
        printf ( "File opening Failed\n" );
    else
        printf ( "File opening Success\n" );

    fclose ( fp );
    return 0;
}
```

Basic File Mode

Mode		Meaning
"r"	"rb"	Read from the beginning of the file
"w"	"wb"	Write from the beginning of the file. If the file exists, its existing contents will be erased
"a"	"ab"	Write from the beginning of the file. If the file does not exist, it is created. If it does exist, all writes occur at the end (append mode)
"r+", "w+", "a+"	"rb+", "wb+", "ab+"	Update mode - both reading AND writing are possible

"r"



Read from the
beginning of the file.

"w"



Write from the
beginning of the file.
If the file exists,
its existing contents
will be erased.

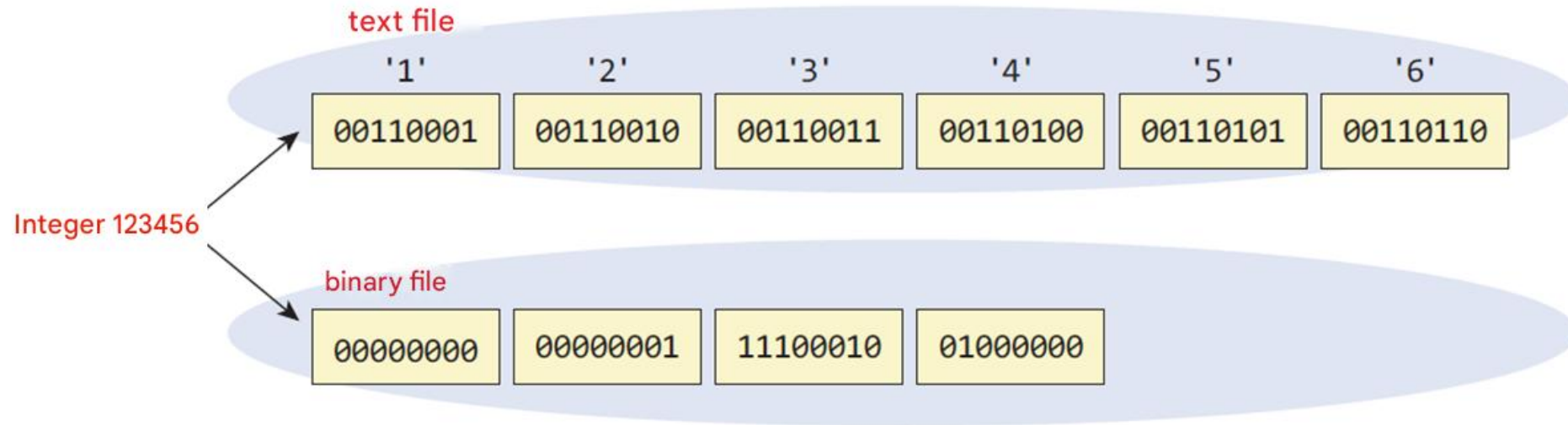
"a"



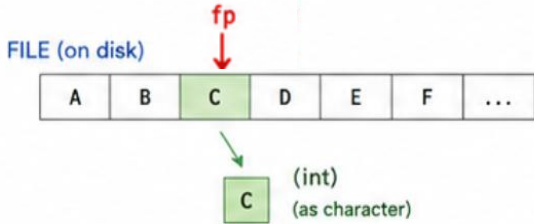
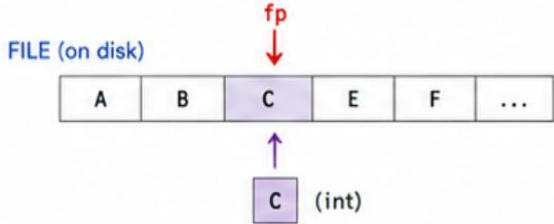
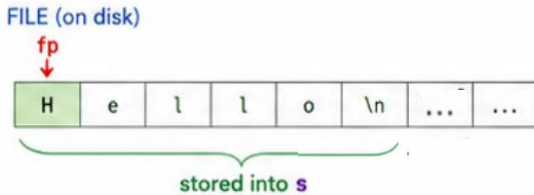
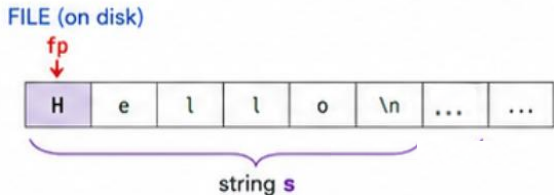
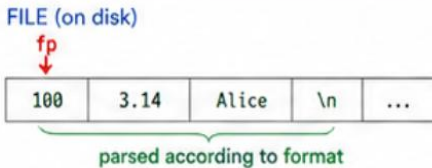
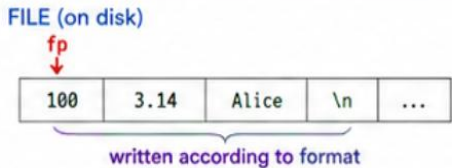
Write from the
beginning of the file.
If the file does not exist,
it is created.

Writing and reading a File

- Difference between text files and binary files
 - *Text file* : All data is converted to ASCII code and saved.
 - *Binary file* : Stores data(raw bytes) exactly as it is represented on a computer.



Writing and reading a File: Text File

Type / Unit	Input function (from file)	What it does (input)	Output function (to file)	What it does (output)
Character unit (one character) 1 byte	<pre>int fgetc(FILE *fp);</pre> Reads one character from the file.		<pre>int fputc(int c, FILE *fp);</pre> Writes one character to the file.	
String unit (string / line) multiple bytes	<pre>char *fgets(char *s, int n, FILE *fp);</pre> Reads a string (up to n-1 chars or until newline or EOF) and stores it in <i>s</i>.		<pre>int fputs(const char *s, FILE *fp);</pre> Writes the string <i>s</i> to the file.	
Formatted input and output	<pre>int fscanf(FILE *fp, const char *format, ...);</pre> Reads formatted data according to <i>format</i>.		<pre>int fprintf(FILE *fp, const char *format, ...);</pre> Writes formatted data to the file.	



FILE *fp

- FILE pointer
- Points to the current position in the file.
- After each read/write, the position moves forward automatically.



Notes

- Text functions (*fgetc*, *fgets*, *fscanf*, etc.) deal with characters.
- Binary functions (*fread*, *fwrite*) deal with raw bytes.
- Use *fscanf*/*fprintf* for formatted text (numbers, words, etc.).



Return value

- *fgetc*, *fputc* : the character read/written (or EOF on end of file)
- *fgets* : *s* (or NULL on error/EOF before any read)
- *fputs*, *fprintf* : non-negative on success, EOF on error
- *fread*, *fwrite* : number of items successfully read/written (may be less than count)

String Unit I/O: fgets() and How It Reads a Line

- fgets(**buffer**, size, fp) reads characters from a file until one of the following happens:
 - 1. A newline character (`\n`) is read
 - 2. size - 1 characters have been read, leaving space for the null terminator
 - 3. End of File (EOF) is reached
- So, fgets() reads one line at a time, INCLUDING the trailing `\n`.

Formatted Input/Output: fprintf(), fscanf()

```
FILE * fp ;
int number, count = 0;
char name[20];
float score, total = 0.0;

// Write student number, name, and grade to the file
while ( 1 )
{
    printf ( "Enter student number, name, grade (negative to end): " );
    scanf ( "%d" , &number);
    if ( number < 0 ) break;
    scanf ( "%s %f" , name, &score);
    fprintf ( fp , "%d %s %f" , number, name, score);
}
fclose ( fp );
```

Example: Read & Write Text File

file_text.c

```
#include <stdio.h>
int writeFile( void )
{
    FILE * fp = NULL;

    fp = fopen ( "sample.txt" , "w" );
    if ( fp == NULL ) {
        printf ("file opening Failed \n" );
    }
    else {
        printf ("file opening Success \n" );
    }

    // 1. save character
    fputc ('a', fp );
    fputc ('b', fp );
    fputc ('c', fp );
    fputc ('\n', fp );

    // 2. save string
    fputs("hello", fp);
    fputs(" world\n", fp);

    // 3. save number
    fprintf(fp, "%d %d %d %.2f", 1, 2, 3, 3.14);

    fclose ( fp );
    return 0;
}
```

```
int readFile() {
    FILE *fp = fopen("sample.txt", "r"); // read mode

    if (fp == NULL) {
        printf("Fail to open\n");
        return 1;
    }

    // 1. read 4 characters
    char ch1 = fgetc(fp);
    char ch2 = fgetc(fp);
    char ch3 = fgetc(fp);
    char ch4 = fgetc(fp);

    // 2. read string (12 character "hello world\n"
    // including white space)
    char str1[10];
    char str2[10];
    fscanf(fp, "%s %s", str1, str2);

    // 3. read 3 integers & 1 real number
    int n1, n2, n3;
    float f;
    fscanf(fp, "%d %d %d %f", &n1, &n2, &n3, &f);

    // print
    printf("\n== readFile ==\n");
    printf("Chars: %c %c %c %c", ch1, ch2, ch3, ch4);
    printf("String: %s %s\n", str1, str2);
    printf("Integers: %d %d %d\n", n1, n2, n3);
    printf("Float: %.2f\n", f);

    fclose(fp);
    return 0;
}
```

```
int readFileByOne( void )
{
    FILE * fp = NULL;
    int c;
    fp = fopen ( "sample.txt" , "r" );
    if ( fp == NULL )
        printf ( "file opening Failed \n");
    else
        printf ( "file opening Success \n");

    printf("\n== readFileByOne ==\n");
    while ((c = fgetc ( fp )) != EOF ) {
        putchar (c);
    }
    fclose ( fp );
    return 0;
}

int readFileByLine( void )
{
    FILE * fp = NULL;
    int SIZE = 100;
    char line[SIZE];

    fp = fopen ( "sample.txt" , "r" );
    if ( fp == NULL )
        printf ( "file opening Failed \n");
    else
        printf ( "file opening Success \n");

    printf("\n== readFileByLine ==\n");
    while (fgets (line, SIZE, fp )) {
        printf ("%s", line);
    }
    fclose ( fp );
    return 0;
}
```

```
void main()
{
    writeFile();
    readFile();
    readFileByOne();
    readFileByLine();
}
```


Read Binary File: fread()

```
size_t fread(void *buffer, size_t size, size_t count, FILE *fp);
```



Return value

- Returns the number of items successfully read.
- May be less than **count** if end-of-file is reached or an error occurs.

- **buffer** : address of memory block where data will be stored
- **size** : size of each item (in bytes). For binary files, 1 byte = 1 raw data (character).
- **count** : number of items to read
- **fp** : FILE pointer (current read position in the file)

FILE (on disk) : raw bytes (character type)

fp
(file pointer)

count items to read

...	1A	3F	7C	A8	00	FF	10	2B	6D	90	11	C3	5E	D4	7A	FE	09	3B	...
-----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

2B

size = 1 byte

fread() reads **count** items (**size** bytes each) from the file starting at **fp**, and stores them into **buffer**.

Memory (buffer)

2B	6D	90	11	C3	5E							...
----	----	----	----	----	----	--	--	--	--	--	--	-----

count items

buffer (must have at least **count** × **size** bytes)

Write Binary File: fwrite()

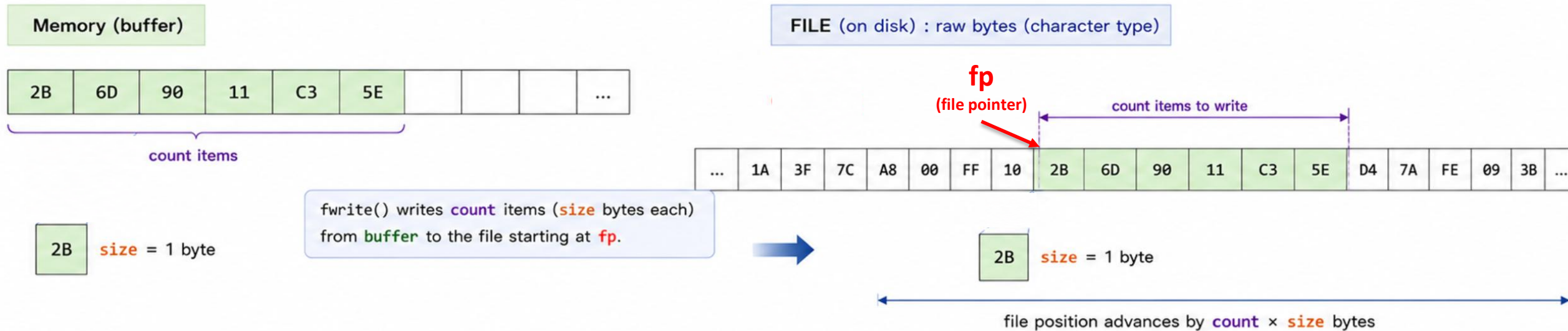
```
size_t fwrite(const void *buffer, size_t size, size_t count, FILE *fp);
```

- **buffer** : address of memory block that contains data to write
- **size** : size of each item (in bytes). For binary files, 1 byte = 1 raw data (character).
- **count** : number of items to write
- **fp** : FILE pointer (current write position in the file)



Return value

- Returns the number of items successfully written.
- May be less than **count** if disk is full or an error occurs.



Note

- **size** is in bytes. Total bytes written = **size** × **count**.
- These functions are used for binary (raw byte) data only.

Example: Read & Write Binary File

file_binary.c

```
#include <stdio.h>

struct Data {
    char c;
    int i;
    float f;
};

int writeFile() {
    // "wb" = write binary
    FILE *fp = fopen("data.bin", "wb");
    if (fp == NULL) {
        printf("Fail to open\n");
        return 1;
    }

    struct Data d = { 'A', 100, 3.14f };

    // save binary of structure
    fwrite(&d, sizeof(struct Data), 1, fp);

    fclose(fp);
    printf("Complete writing.\n");
    return 0;
}

int readFile() {
    // "rb" = read binary
    FILE *fp = fopen("data.bin", "rb");

    if (fp == NULL) {
        printf("Fail to open\n");
        return 1;
    }

    struct Data d;

    // read binary of structure
    fread(&d, sizeof(struct Data), 1, fp);

    printf("[read data]\n");
    printf("\tchar: %c\n", d.c);
    printf("\tint: %d\n", d.i);
    printf("\tfloat: %.2f\n", d.f);

    fclose(fp);
    return 0;
}

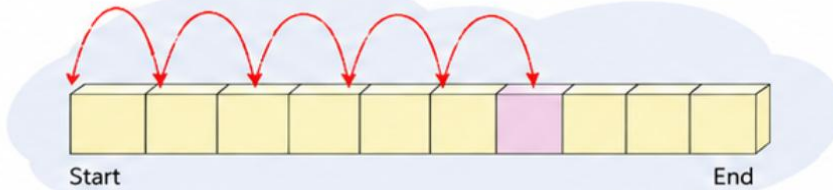
void main()
{
    int ret = writeFile();
    if(ret == 0){
        ret = readFile();

        if(ret == 0){
            printf("SUCCESS!!");
        } else {
            printf("FAIL!!");
        }
    }
}
```

Random Access File: fseek()

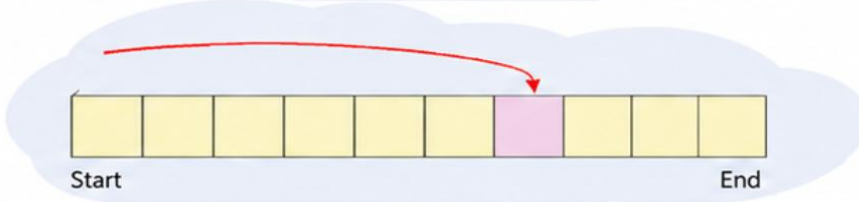
- **Sequential access** Method : How to read or write data sequentially from the beginning of a file
- **Random access** Method : How to read and write from any location in the file

Sequential access file



Read/Write moves in order from the beginning.

Random access file



Read/Write can jump to any location in the file.

Syntax `fseek()`

`int` `fseek`(FILE *fp, long offset, int origin);

yes

fp : FILE pointer
offset : number of bytes to move
origin : reference point for offset

origin	value	reference point	description
SEEK_SET	0	Start (of file)	Start of file
SEEK_CUR	1	Current position	Current location (fp position)
SEEK_END	2	End (of file)	End of file

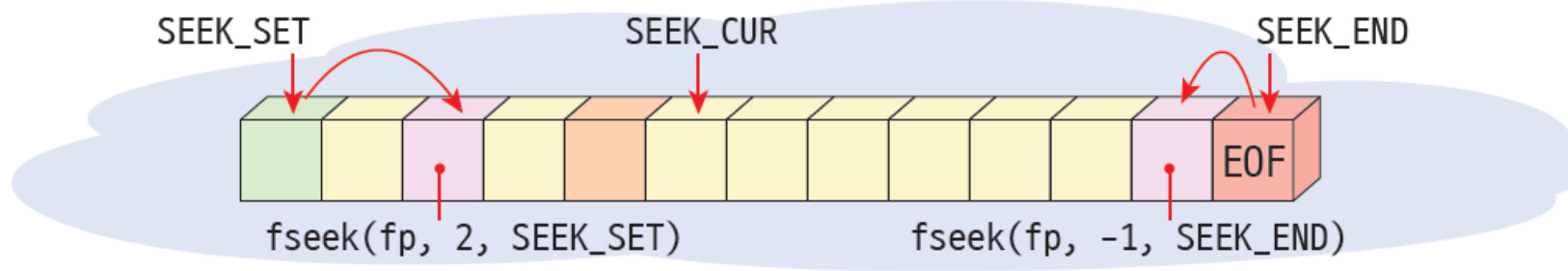


Note

- `fseek()` repositions the file pointer (fp) to the new location.
- It does not read or write any data.

Example : `fseek(fp, 100, SEEK_SET);` // move to byte 100 from start
`fseek(fp, -20, SEEK_CUR);` // move back 20 bytes
`fseek(fp, 0, SEEK_END);` // move to end of file

Random Access File: fseek(), ftell(), feof()



Syntax: `ftell()`

Returns the current position of the file pointer

예 `long ftell(FILE *fp);`

Syntax: `feof()`

Returns whether the end of file has been reached

예 `int feof(FILE *fp);`

Q & A

Lab & Quiz: right after this - see you in the practice session!